Document Number: P0828R1
Date: 2018-06-25
Reply-to: John McFarlane
Audience: SG6, SG12, SG14

Elastic Integers

1. Introduction

This paper proposes a numeric type, `elastic_integer`, which returns widened results from common arithmetic operations in order to prevent out-of-range errors. It is highly composable and can be combined with other types such as overflow-trapping and fixed-point numeric types to provide arithmetic with all of the efficiency of integers and some of the benefit traditionally enjoyed by dynamic types such as bignum and floating-point.

2. Motivation and Scope

2.1 Out-of-Range Integer Arithmetic

Fundamental integer types are usually successful in representing quantities. When they are unsuccessful, it is often due to operations producing results with out-of-range values.

```
auto positive_unsigned_overflow = UINT_MAX + 1;  // 0U
auto positive_signed_overflow = INT_MAX + 1; // undefined behavior
auto negative_unsigned_overflow = 0U > -1;   // false
```

2.2 Preventing Errors

It is difficult and costly to detect and correct such errors. As always, prevention is better than cure:

```
auto positive_unsigned_ok = static_cast<unsigned long long>(UINT_MAX) + 1;
auto positive_signed_ok = static_cast<long long>(INT_MAX) + 1;
auto negative_unsigned_ok = static_cast<long long>(0U) > -1;
```

This solution places the responsibility for managing range firmly with the user, a task which is made especially onerous for two reasons.

Firstly, the solution is not generic: in a setting where the integer type is a template parameter, the user cannot easily call upon the next widest integer by type.

Secondly, difference in the width of integers on different architectures means calling upon the next widest integer type is not even foolproof in non-generic code. Indeed, the above casts may not increase range on platforms where `sizeof(long long) == sizeof(int)`. (Fixed-width integer types do not completely solve this problem as they are aliases to types which may be wider than advertised.)

2.3 Portable and Generic

Portable, generic code for avoiding integer overflow requires facilities to manipulate two properties of integers directly affecting range: signedness and number of digits. `is_signed`, `make_signed` and `make_unsigned` supported the manipulation of signedness for fundamental integers types only. The manipulation of the number of digits is even less well supported.

P0102 and P0675 both propose facilities for requesting an integer by its width or its number of digits, respectively. P0102 is limited to fundamental arithmetic types whereas P0675 caters for user-defined types.

P0675's `num_digits_v` and `set_num_digits_t` components make it possible to widen correctly for arithmetic operations:

```
template<typename Operand>
auto multiply(Operand a, Operand b) {
    // Get the number of digits in the input type.
    constexpr auto operand_digits = num_digits_v<Operand>;

    // Results of multiplication contain twice the number of digits.
    constexpr auto result_digits = operand_digits * 2;
```

```
    // Use this to determine the result type.
    using result_type = set_num_digits_t<Operand, result_digits>;

    // Promoted operands before operation.
    return static_cast<result_type>(a) * static_cast<result_type>(b);
}
```

Two limitations with this approach are:

1. It requires the use of a named function instead of arithmetic operators.
2. For operations such as addition and comparison, only a single extra bit is required in the form of a digit or the sign bit. But fundamental arithmetic types generally only come in widths that are powers of two. So widening by a single bit entails widening by a factor of two.

2.4 Usability

Both of the above problems can be solved with a numeric type, `elastic_integer` (reference implementation, live demo) that parametrizes signedness and number of digits:

```
template<int Digits, class Narrowest = int>
class elastic_integer;
```

Where:

- `Digits` specifies the number of binary digits used to express magnitude (excluding sign bit). It corresponds directly to the `num_digits_v` variable template and `set_num_digits_t` class template from P0675. `Digits` must be a positive value.
- `Narrowest` specifies the narrowest type used to store the value. If `Digits` exceeds the capacity of `Narrowest`, a wider type is substituted to ensure that the value can be represented.

For two's complement signed types, the most negative number is not in the range allowed by the type. This avoids certain edge cases, e.g. where `-INT_MIN` exceeds `INT_MAX`, thus requiring an entire extra digit.

`elastic_integer` automates the work of tracking range. Thus, the user is less likely to experience out-of-range errors:

```
auto d(elastic_integer<15> x, elastic_integer<15> y)  // [-32767..32767]
{
  // When two values are multiplied, result is the sum of the `Digits` parameters.
  auto xx = x*x, yy = y*y;  // elastic_integer<30>

  // When two values are summed, result is the `Digits` parameter plus one.
  auto dd = xx + yy;    // elastic_integer<31>

  // A square root operation almost halves the number of digits required
  auto d = square_root(dd); // elastic_integer<31> in range [0..46339]

  // so we know it's safe to cast to a narrower type.
  return elastic_integer<16, unsigned>(d);
}
```

2.5 Initialization

`elastic_integer` can be initialized from any integer type which employs the customization points laid out in P0675. For example:

```
auto a = elastic_integer(123);  // commonly deduced as elastic_integer<31>
auto b = elastic_integer(UINT64_C(4096));  // commonly deduced as elastic_integer<64, unsigned>
```

With general-purpose constant value type such as those proposed in P0377 and P0827 an optimized `Digits` parameter can be determined:

```
auto a = elastic_integer(constant<123>());  // deduced as elastic_integer<7>
auto b = elastic_integer(constant<4096>());  // deduced as elastic_integer<13>
```

The addition of a user-defined literal for generating constant value types further improves terseness:

```
auto a = elastic_integer(123static);  // deduced as elastic_integer<7>
auto b = elastic_integer(4096static);  // deduced as elastic_integer<13>
```

3 Impact On the Standard

This proposal adds header, `<elastic_integer>`, containing class template, `elastic_integer` and a collection of operator overload function templates which only participate in overload resolution when either of the operands is of type, `elastic_integer`.

The usefulness of `elastic_integer` is greatly enhanced with a number of additions. In particular, P0675 allows `elastic_integer` to be composed with other types and P0827 adds a number of facilities to `elastic_integer` which make it more efficient and usable.

4. Design Decisions

The design of `elastic_integer` arises from observations of the `fixed_point` type from P0037 when compared with the fixed-point types proposed by Lawrence Crowl in P0106. P0106's fixed-point types -- not only approximate real numbers using integer arithmetic but also -- address a variety of concerns related to accuracy and safety such as rounding and out-of-range errors. In response to these observations, P0554 argues for an alternative approach in which individual concerns are addressed by individual numeric components and those components are used to compose a composite type which solves the full list problems identified in P0106.

One such problem addressed in P0106 is that of preventing out-of-range conditions. The solution there is the same *elasticity* property adopted by `elastic_integer`. The difference is that `elastic_integer` does nothing else but solve this single problem.

4.1 Composability

4.1.1 Composite Type with Run-Time Overflow Checks

`elastic_integer` offers little protection against out-of-range errors caused by narrowing or compound operations that increases the magnitude of the value:

```
elastic_integer<10> a = 1023;   // within range; OK
auto b = elastic_integer<9>(a); // narrowing cast exceeds range; undefined behavior
++ a;   // increment exceeds range; undefined behavior
a = -1024;  // narrowing assignment exceeds range; undefined behavior
```

Such errors cannot be prevented using the type system alone. Either the user has to take care to avoid these errors or run-time checks must be performed. Fortunately, this can be remedied by combining `elastic_integer` with a so-called 'safe' integer type, e.g.:

```
// a numeric component that traps out-of-range errors
template<class T>
overflow_integer;

overflow_integer<elastic_integer<10>> a = 1024; // trap!
auto aa = a*a;    // no need for run-time check: num_digits_v<decltype(a*a)> is wide enough to hold the res
```

(This composite type is said to have a numeric component nesting depth of 2 because one numeric component is nested within another. `elastic_integer<10, int>` has a nesting depth of 1 and `int` has a nesting depth of 0.)

4.1.2 Composite Type with Real Number Approximation

P0037 introduces a fixed-point type which uses integers to approximate real numbers:

```
template<class Rep, int Exponent>
class fixed_point;
```

Fixed-point arithmetic increases the opportunity for out-of-range errors. In the following example on a system where `int` is 32 bits, a 31-digit integer is expected to store a 53-bit value, resulting in UB.

```
auto kibi = fixed_point<int32_t, -16>(1024); // 2^26
auto mebi = kibi * kibi;   // fixed_point<int, -32>; value: 2^52
```

The problem here is with the backing type used to represent the fixed-point value -- not the fixed-point arithmetic per se. But the fact that fixed-point types often use more of their allotted capacity exacerbates the issue. So replacing `int` with `elastic_integer` produces a composite type that prevents the error:

```
template<int Digits, int Exponent>
using elastic_fixed_point = fixed_point<elastic_integer<Digits>, Exponent>;

auto kibi = elastic_fixed_point<31, -16>(1024); // stores value 2^26
auto mebi = kibi * kibi;   // elastic_fixed_point<62, -32> stores value: 2^52
```

The benefits of the composite, `elastic_fixed_point` is significant. Not only does it eliminate the risk of both overflow *and* underflow, but it does it with the minimum of integer arithmetic. A modern optimizing compiler reduces the above multiplication to a single integer operation:

```
auto kibi = 1024*65536;
auto mebi = int64_t{kibi}*kibi;
```

4.1.3 Composing for Compactness

The default type for representing integer quantities is `int`. It is the default for `Narrowest` for the same reasons. This means that even a one-digit type (`elastic_integer<1>`), is as wide as `int`. When storage matters more than performance, `int` is the wrong choice. For example:

```
struct date {
  elastic_integer<15, int8_t> year;   // [-32767..32767]
  elastic_integer<4, uint8_t> month;  // [0..15]
  elastic_integer<5, uint8_t> day;    // [0..31]
};

constexpr auto epochalypse = date{2038, 1, 19};
constexpr auto end_of_ice_age = date{-10000, 7, 19};
```

Typically, the sizes of the three member variables of `date` are 2, 1 and 1 bytes respectively and most users can assume that `sizeof(date)==4`.

4.2 Heterogeneous Operators

The usability of numeric component types in arithmetic expressions is greatly affected by the set of types accepted by their operators. In the case of `elastic_integer`, being able to perform operations between an `elastic_integer` type and a non-`elastic_integer` type is safe and convenient:

```
auto a = elastic_integer<2>(2) + 2;  // equivalent to elastic_integer<2>(2) + elastic_integer(2)
```

A summary of the rules for binary operators taking a single `elastic_integer` operand are as follows:

1. If the non-`elastic_integer` operand is a static numeric component (e.g. `int` or `overflow_integer<int>`):
    1. If the left-hand operand is less deeply nested than the right-hand operand, then the left-hand operand is converted to an equivalent type of the same class template as the right-hand operand and the operation is performed on the converted operand and the right-hand operand.
    2. If the left-hand operand is more deeply nested than the right-hand operand, then the right-hand operand is converted to an equivalent type of the same class template as the left-hand operand and the operation is performed on the left-hand operand and the converted operand.
    3. If both operands are equally deeply nested, no operator matches and a compiler error is emitted.
2. If the non-`elastic_integer` operand is a dynamic numeric component (e.g. floating-point or bignum), the `elastic_integer` operand is converted to the dynamic numeric component and operation is performed on the two dynamic numeric components.

These rules should be applicable to all numeric components. However, the rules for when both operands are the same numeric component are specific to that component. In the case of `elastic_integer` the rules vary depending on the operator.

4.3 Open Issues

4.3.1 Zero Digits

It is uncertain whether allowing `elastic_integer<0>` is wise or even possible.

For types instantiated from unsigned fundamental integers, e.g. `elastic_integer<0, unsigned>`, only the value zero is possible. Whether such a type would require any storage at all is unclear. If not, it would be a special case because storage is normally rounded up to the width of the `Narrowest` parameter. `elastic_integer<0, void>` might be a way to express a zero-sized type. If a zero-sized integer becomes standardized, a zero-digit elastic type would suddenly make a lot more sense and zero storage would be expected.

For types instantiated from signed fundamental integers, there is the added question of the sign bit. It might be expected but would certainly be of little use. Even on two's complement systems where a zero-digit signed integer has the range [0..-1], the rule forbidding the most negative number would reduce the range to zero.

4.3.2 Bikeshedding

It is not at all clear that elasticity is an appropriate metaphor for this auto-widening behavior.

5 Technical Specification

5.1 Header `<elastic_integer>` synopsis

```
namespace std {
  // class template elastic_integer
  template<int Digits = num_digits_v<int>, class Narrowest = int> class elastic_integer;

  // unary operators
  template<int Digits, class Narrowest>
    constexpr elastic_integer<Digits, Narrowest>
      operator+(const elastic_integer<Digits, Narrowest>&);
  template<int Digits, class Narrowest>
    constexpr elastic_integer<Digits, std::make_signed_t<Narrowest>>
      operator-(const elastic_integer<Digits, Narrowest>&);

  // binary operators (see 4.1 for return types)
  template<int Digits1, class Narrowest1, int Digits2, class Narrowest2>
    constexpr auto
      operator@(const elastic_integer<Digits1, Narrowest1>&, const elastic_integer<Digits2, Narrowest2>&);
  template<int Digits, class Narrowest, class T>
    constexpr auto
      operator@(const elastic_integer<Digits, Narrowest>&, const T&);
  template<class T, int Digits, class Narrowest>
    constexpr auto
      operator@(const T&, const elastic_integer<Digits, Narrowest>&);

  // pre/post increment/decrement
  template<int Digits, class Narrowest>
    constexpr elastic_integer<Digits, Narrowest>&
      operator++(elastic_integer<Digits, Narrowest>&);
  template<int Digits, class Narrowest>
    constexpr elastic_integer<Digits, Narrowest>
      operator++(elastic_integer<Digits, Narrowest>&, int);
  template<int Digits, class Narrowest>
    constexpr elastic_integer<Digits, Narrowest>&
      operator--(elastic_integer<Digits, Narrowest>&);
  template<int Digits, class Narrowest>
    constexpr elastic_integer<Digits, Narrowest>
      operator--(elastic_integer<Digits, Narrowest>&, int);
```

```
  // compound assignment operators
  template<int Digits, class Narrowest, class T>
    constexpr elastic_integer<Digits, Narrowest>&
      operator@=(elastic_integer<Digits, Narrowest>&, const T&);

  // numeric traits (see P0554 for details)
  template<int Digits, class Narrowest>
    struct digits<elastic_integer<Digits, Narrowest>>;
  template<int Digits, class Narrowest, int MinNumBits>
    struct set_digits<elastic_integer<Digits, Narrowest>, MinNumBits>;
  template<int Digits, class Narrowest>
    constexpr typename elastic_integer<Digits, Narrowest>::rep
      to_rep(const elastic_integer<Digits, Narrowest>&);
  template<int Digits, class Narrowest, class Rep>
    struct from_rep<elastic_integer<Digits, Narrowest>, Rep>;
  template<int Digits, class Narrowest, class Value>
    struct from_value<elastic_integer<Digits, Narrowest>, Value>;
  template<int ShiftDigits, int ScalarDigits, class ScalarNarrowest>
    struct shift<ShiftDigits, 2, elastic_integer<ScalarDigits, ScalarNarrowest>>;
}
```

5.2 Class template `elastic_integer`

```
namespace std {
  template<int Digits, class Narrowest>
  class elastic_integer {
    public:
      using rep = set_num_digits_t<Narrowest, max(Digits, num_digits_v<Narrowest>)>;
    private:
      rep rep_;  // exposition only
    public:
      elastic_integer() = default;
      template<class N> constexpr elastic_integer(const N&);
      template<class N> elastic_integer& operator=(N s);
      template<class N> explicit constexpr operator N() const;
  };
}
```

6 Discussion

This section explores feedback raised in review.

Q: Why might the signedness of the result of unary `operator+` be signed, even when the input is unsigned?

A: The reason to consider returning a signed result is symetry. When using native integers, `-1U` is `0xffffffffU`. In other words, both signed and unsigned fundamental types return the same signedness for unary plus and minus. In the case of unsigned integers, it gives us the much-lamented off-by-4.3-billion error. However, there is little reason to make the result signed in the case of unary `operator+`.

Q: If converting from unsigned to signed, shouldn't the number of bits be increased by one in case the unsigned number coming in can't fit?

A: The incoming number is guaranteed to fit for two reasons: Firstly, `Digits` does not include the sign bit so the type will be a bit wider but `Digits` will remain unchanged. Secondly, signed `elastic_integer` has a symetric range around zero, i.e. the most negative number is not a valid value. (See section 2.4 for more details.)

Q: Why don't post and pre-increment, post and pre-decrement and compound assignment operators return wider types?

A: `elastic_integer` returns a wider type when it is opportune to do so but it cannot perform miracles. The pre/post and compount assignment operators mutate the value passed to them so returning a wider type is either partially or totally futile. (See section 4.1.1 for details.)

Q: Why are mutating operators such as pre-increment and compound assignment marked as `constexpr`.

A: So they can be used in constant expressions. (Example)

7 Acknowledgements

Thanks to Arthur O'Dwyer for advice on customization points.